

Architecture Reference

How the marketing site, content engine, and indexing engine fit together — plus a phased rollout playbook for getting millions of programmatic-SEO pages indexed by Google.

1.

The site has three layers

Every page lives in exactly one of these layers, and each layer has a different visual style, audience, and indexing goal.

Layer	What it is	Lives at
1. Marketing site	The cinematic home page — luxury aviation brand. Force-dark, full-bleed 3D globe and plane scenes. No navigation.	3D globe and plane scenes . No navigation.
2. Content engine	Programmatic SEO pages: route pages, airport pages, coming-soon pages, demo. Every page wears the Site Kit template.	pages/demo . Every page wears the Site Kit template.
3. Indexing engine	Not visible to users — sitemaps, robots, IndexNow key file, GSC verification, RSS, admin status endpoints, API.	sitemap , robots , IndexNow , GSC , RSS , admin , status , endpoints , API .

Project map (annotated)

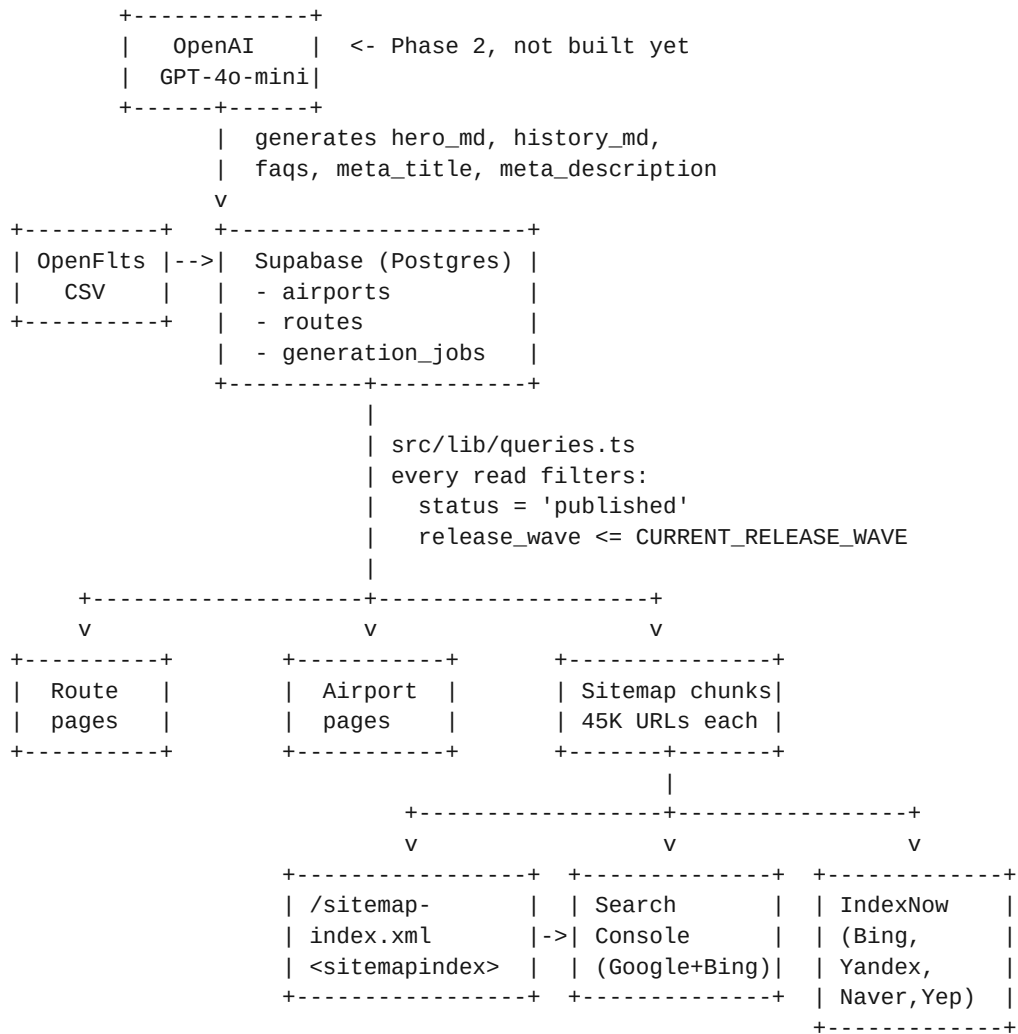
The whole codebase fits on one page. Layer boundaries are explicit in the folder structure — the (content) route group is the entire content engine, and indexing files cluster at the app root.

```
heritage_flights/
|
|-- supabase/migrations/
|   |-- 0001_init.sql           # airports, routes, generation_jobs, RLS
|   `-- 0002_indexing.sql      # + release_wave, published_at, triggers
|
|-- scripts/
|   |-- seed-openflights.ts    # Supabase <- OpenFlights CSV (idempotent)
|   |-- build-architecture-pdf.py # This document's generator
|   `-- data/                  # local CSVs (7K airports, 67K routes)
|
|-- src/app/
|   |
|   |-- layout.tsx             # <html>, theme init, GSC meta, sitemap link
|   |-- globals.css            # Backpack tokens, dark variant
|   |-- page.tsx               # LAYER 1 (marketing) – force-dark, scoped
|   |
|   |-- sitemap.ts             # LAYER 3 – small (static pages only)
|   |-- sitemap-index.xml/     # LAYER 3 – <sitemapindex>, submit to GSC
|   |   |-- route.ts
|   |-- robots.ts             # LAYER 3 – allow/disallow, blocks SEO bots
|   |-- [indexnowKey]/         # LAYER 3 – /<key>.txt verification (root)
|   |   |-- route.ts
|   |
|   `-- (content)/             # LAYER 2 – route group; doesn't affect URL
|       |
|       |-- layout.tsx         # injects <SiteNav /> on all content pages
|       |
|       |-- routes/
|       |   |-- sitemap.ts     # generateSitemaps() -> /routes/sitemap/[id].xml
|       |   `-- [origin]/[destination]/[slug]/page.tsx
|       |
|       |-- airports/
|       |   |-- sitemap.ts     # generateSitemaps() -> /airports/sitemap/[id].xml
|       |   `-- [iata]/page.tsx
|       |
|       |-- hotels/page.tsx    # coming-soon
|       |-- cars/page.tsx     # coming-soon
|       `-- demo/mumbai-to-go/ # hand-written sample of generated content
|           `-- page.tsx
|
|-- src/components/
|   |-- layout/SiteNav.tsx     # logo, links, region, theme toggle, login
|   |-- layout/ComingSoon.tsx # reusable empty-state for unbuilt surfaces
|   |-- theme/
|   |   |-- theme-init.tsx    # inline <head> script, runs before paint
|   |   `-- theme-toggle.tsx  # sun/moon button
|   `-- flights/
|       |-- FlightSearchWidget.tsx # Skyscanner-style booking form
|       |-- RouteHeader.tsx, BookingCTA.tsx
|       `-- Markdown.tsx, FAQs.tsx, OtherFlights.tsx
```

```
`-- src/lib/
  |-- env.ts                # validated env access (throws on missing)
  |-- supabase.ts           # public + admin (service-role) clients
  |-- database.types.ts     # row + insert types per table
  |-- openai.ts             # client (Phase 2 content worker)
  |-- queries.ts            # all DB reads, ALL wave-gated
  |-- slug.ts               # canonical URL builders
  |-- affiliate.ts          # Skyscanner deep-link with associate ID
  `-- indexnow.ts           # POST /IndexNow with batched URLs
```

Data flow

Two inputs (a public dataset and an LLM) feed one source of truth (Supabase). Every public surface — pages, sitemaps, RSS — derives from that, with the wave gate as a single filter applied at read time.



Indexing engine — five concrete pieces

Each piece is a small file. Together they implement the mega-scale SEO pattern recommended by Google: chunked sitemaps, explicit crawl-budget rules, IndexNow side-channel for Bing, and a single env-var release switch.

Piece	Where it lives	Job
Wave gate	<code>env.currentReleaseWave + .lte("release-wave")</code>	One env-var controls which DB rows are publicly visible. Bump CURR
Chunked sitemaps	<code>(content)/routes/sitemap.ts</code> <code>(content)/airports/sitemap.ts</code>	<code>generateSitemaps()</code> fans each segment into 45K-row chunks. At 67K
Sitemap-index	<code>app/sitemap-index.xml/route.ts</code>	Hand-built <sitemapindex> XML — the single URL you submit to GSC
IndexNow	<code>lib/indexnow.ts</code> + <code>app/[indexnowKey]/route.ts</code>	Push up to 10K URLs per call to Bing, Yandex, Naver, Yep. Google n
Crawl-budget hygiene	<code>app/robots.ts</code>	Explicit Disallow on <code>/api/</code> , <code>/i/</code> , <code>/_next/</code> , <code>/admin</code> , <code>/demo/</code> . Blocks AhrefsBo

5.

The wave gate, in detail

Phased publishing is the single biggest indexing accelerator after buying an aged domain. The wave gate makes it a one-line config change instead of a deploy.

Schema (migration 0002)

```
-- Both columns default to 1 – existing rows pass the gate at wave 1.
ALTER TABLE routes
  ADD COLUMN release_wave integer NOT NULL DEFAULT 1,
  ADD COLUMN published_at timestamptz;

-- Same on airports.
-- A trigger stamps published_at when status flips to 'published'.
```

Read path (src/lib/queries.ts)

```
const CURRENT_WAVE = env.currentReleaseWave;

// Every public-facing query applies BOTH filters:
await supabase.from('routes')
  .select('*')
  .eq('origin_iata', o).eq('destination_iata', d)
  .eq('status', 'published') // generated content exists
  .lte('release_wave', CURRENT_WAVE) // wave unlocked
  .maybeSingle();
```

Phased rollout schedule

Wave	When	Pages live	Action
1	Day 1	~5,000	CURRENT_RELEASE_WAVE=1 (default)
2	Week 2	~20,000	Bump env var; redeploy
3	Week 6	~70,000	Bump env var
4	Month 2-3	~2,00,000	Bump env var
5	Month 3-4	~5,00,000	Bump env var
6	Month 4-6	~25,85,000	Final phase – all routes unlocked

Routes generated into a future wave stay literally invisible: not in any sitemap, not internally linked, 404 if hit directly. Google never sees a spam signal from "millions of pages appeared overnight".

Tactic from the brief -> implementation

A direct mapping from the 12 acceleration tactics in the indexing brief to where each one is implemented (or where it lives outside code).

#	Tactic	Where in the codebase
1	Buy aged domain (DA 30+)	External — buy from Odys / ExpiredDomains. Point NEXT_PUBLIC_SITE_URL at it.
2	Phased publishing	release_wave column + CURRENT_RELEASE_WAVE env var (see section 5).
3	Clean XML sitemaps (50K chunks)	(content)/{routes,airports}/sitemap.ts via generateSitemaps().
4	IndexNow for Bing/Yandex	lib/indexnow.ts + app/[indexnowKey]/route.ts.
5	Server speed under 200ms	Next.js 16 ISR (revalidate=86400) + Vercel edge / Cloudflare CDN at deploy.
6	Internal linking architecture	components/flights/OtherFlights.tsx — every route page links 8 siblings.
7	Backlinks from Day 1	External — directories, guest posts, HARO. No code.
8	GSC submission	Submit /sitemap-index.xml. GSC token via NEXT_PUBLIC_GOOGLE_SITE_VERIFICATION_TOKEN.
9	Server-side rendering	Native — Next.js App Router with generateStaticParams + revalidate.
10	Eliminate crawl waste	app/robots.ts (disallows) + slug-mismatch redirect in route page.
11	Google ping / RSS	PENDING — /feed.xml route to be added next.
12	Social signals	External — manual sharing pipelines.

What ships today / what is still pending

A scoreboard of the indexing engine as of today.

Built and verified

Item	Path
Wave gate columns + triggers	supabase/migrations/0002_indexing.sql
Wave-aware queries	src/lib/queries.ts
Chunked sitemap: routes	src/app/(content)/routes/sitemap.ts
Chunked sitemap: airports	src/app/(content)/airports/sitemap.ts
Sitemap-index <sitemapindex> XML	src/app/sitemap-index.xml/route.ts
IndexNow client	src/lib/indexnow.ts
IndexNow key-file route	src/app/[indexnowKey]/route.ts
GSC verification meta tag	src/app/layout.tsx
robots.txt with disallows	src/app/robots.ts
env validation + admin token slot	src/lib/env.ts

Pending

Item	Notes
Move IndexNow route under /i/	Today it sits at the root catch-all. Risk: shadows other paths if any unexpected URL is hit. M
RSS feed at /feed.xml	Latest 50 published routes. Lets Google discover new content via PubSubHubbub. Tactic 1
/api/admin/indexing-status JSON	Returns counts: pages in current wave, pending generation, last IndexNow batch result. Ga
docs/INDEXING.md operational playbook	Step-by-step: bump wave, run IndexNow batch, monitor GSC. To be written; this PDF is a
Phase 2: OpenAI content worker	Cron / route handler that picks pending routes from generation_jobs, calls GPT-4o-mini with
Final build verification	Run npm run build after the items above land, confirm all routes still resolve and sitemaps g

Footer

Heritage Flights · Architecture Reference · generated from scripts/build-architecture-pdf.py · regenerate any time with
python scripts/build-architecture-pdf.py.